# AI SHOPPING AGENT

Practical Engineering Assessment

### Candidate assignment

| Duration | Language | Submission |
|---|---|---|
| 6–8 hours | Python | Git repository |
| **Difficulty** | **Focus** | **Environment** |
| Intermediate–Advanced | Agent engineering | Local mock environment |

## 1. Objective

Build an autonomous AI shopping agent that receives a customer's request, searches a product environment, evaluates available products, and returns the best valid recommendation. The agent must use tools, reason about requirements, maintain state, handle unexpected situations, and produce reliable results.

The goal is not simply to search a database and return the closest matching product. The submission should demonstrate practical agent behavior and sound engineering decisions.

## 2. Customer request

Support a request such as: *"I need a laptop for software development. My maximum budget is $1,200. It must have at least 32GB RAM and 1TB SSD storage. I prefer good battery life and a lightweight design. Compare the best options and recommend one."*

| Parameter | Requirement | Type |
|---|---|---|
| Category | Laptop | Mandatory |
| Maximum price | $1,200 | Mandatory |
| Minimum RAM | 32GB | Mandatory |
| Minimum storage | 1TB SSD | Mandatory |
| Battery life | Preferably 10+ hours | Preference |
| Weight | Preferably under 1.6kg | Preference |
| Purpose | Software development | Context |

**Critical rule:** Separate mandatory constraints from preferences. A product with only 16GB RAM must never be recommended as a valid match, even if it is cheaper or highly rated.

## 3. Product environment

Create or use a mock catalog containing at least 30 products. Each record should include fields equivalent to this example; you may extend the schema.

```
{ "product_id": "LAPTOP_001", "name": "Example Laptop", "category": "laptop", "price": 1099, "ram_gb": 32,
"storage_gb": 1024, "storage_type": "SSD", "battery_hours": 11, "weight_kg": 1.45, "rating": 4.6,
"availability": true, "description": "Laptop for professional development" }
```

Include problematic records: products over budget; insufficient RAM or storage; out-of-stock products; missing specifications; contradictory descriptions and structured fields; and highly rated products that are poor matches.

## 4. Required agent responsibilities

| Stage | Responsibility |
|---|---|
| Requirement parsing | Convert natural language into structured constraints, preferences and context. |

| Stage | Responsibility |
|---|---|
| Planning / orchestration | Decide which actions and tools are needed; track task progress. |
| Tool execution | Search, inspect and verify products using available tool interfaces. |
| Validation | Enforce mandatory constraints deterministically; do not rely only on LLM judgment. |
| Ranking / decision | Rank valid products using preferences and explain trade-offs. |
| Final response | Return the best valid recommendation, alternatives, and limitations or clarifying questions. |

A single-agent design is acceptable. A multi-agent design is also acceptable if it provides a clear, measurable benefit. Do not add agents merely for complexity.

## 5. Required tools

Provide these mock tools. The agent should invoke them through a defined interface rather than bypassing the tool layer.

| Tool | Purpose |
|---|---|
| search_products | Search the product catalog. |
| get_product_details | Retrieve complete product information. |
| check_availability | Verify current stock status. |
| compare_products | Compare selected products. |
| get_reviews | Retrieve review information. |

**Dynamic-tool challenge:** Supply tool names, descriptions and parameter schemas through the environment. During the live assessment, introduce an unfamiliar tool such as `inventory_lookup(product_code, region, requested_quantity)`. The candidate should inspect its schema and adapt without hardcoding the original tool's name or parameters.

## 6. Mandatory technical capabilities

**Natural-language understanding:** Extract structured constraints and preferences.
**Tool calling:** Select tools, pass valid arguments and interpret responses.
**Dynamic adaptation:** Inspect unfamiliar tool schemas and use them correctly.
**State management:** Track the request, searches, rejected products, reasons and pending checks.
**Error handling:** Handle timeouts, invalid arguments, empty results and missing data.
**Decision-making:** Enforce hard constraints and rank only valid products.
**Evaluation:** Measure performance with repeatable tests and report failures.

## 7. Test scenarios

Run at least the following scenarios. Keep some cases hidden until evaluation.

| Test | Scenario | Expected behavior |
|---|---|---|
| 1 | Normal laptop request | Find and recommend valid products. |
| 2 | No product meets every hard constraint | State that no valid match exists; do not invent one. |
| 3 | Product has insufficient RAM | Reject it as invalid. |
| 4 | Product is out of stock | Exclude it or clearly identify it as unavailable. |
| 5 | Price changes during the task | Recheck price before recommending. |
| 6 | Product information is missing | Investigate or exclude safely; do not assume. |
| 7 | Search tool fails | Recover safely or report the failure. |
| 8 | Customer changes budget | Update requirements and rerun relevant steps. |
| 9 | Conflicting customer requirements | Ask a targeted clarification question. |
| 10 | Tool schema changes | Adapt to the new tool interface. |

## 8. Required deliverables

1. Complete Python source code.
2. README with installation and execution instructions.
3. Architecture diagram.
4. Product catalog and mock tools.
5. At least 30 test cases, including expected outcomes.
6. Test results, success rate and failure analysis.
7. Logs showing tool calls and agent decisions.
8. Explanation of major architectural decisions.
9. Known limitations and proposed improvements.

The evaluator must be able to run the project locally. Include exact commands and required environment variables. Do not include API keys or secrets in the repository.

## 9. Evaluation scorecard

| Area | Points | Score |
|---|---|---|
| Python and software engineering | 15 | ____ |
| Agent architecture | 10 | ____ |
| Natural-language understanding | 10 | ____ |
| Tool calling | 10 | ____ |
| Dynamic tool adaptation | 15 | ____ |
| State management | 10 | ____ |
| Error handling | 10 | ____ |
| Product validation and decision-making | 10 | ____ |
| Evaluation and testing | 5 | ____ |
| Documentation and explanation | 5 | ____ |
| **Total** | **100** | **____ / 100** |

**85–100:** Strong candidate; consider independent development ownership.
**70–84:** Potentially suitable with initial supervision.
**55–69:** Junior-level capability; substantial guidance needed.
**Below 55:** Not currently suitable for this responsibility.

## 10. Live technical defense

After submission, conduct a 30–45 minute live session. Ask the candidate to modify the code in front of you. This helps establish whether the candidate understands and owns the implementation.

**1. Change a tool schema:** Change the product-search tool's schema. Ask the candidate to adapt the agent.
**2. Introduce an invalid product:** Add a product that satisfies every requirement except RAM. Verify that the agent rejects it.
**3. Simulate API failure:** Make the availability API fail. Ask the candidate to implement safe recovery.
**4. Change the customer's budget:** Change the budget during the conversation. Verify that the agent updates its state and decisions.
**5. Diagnose a failed evaluation:** Show a failed test. Ask the candidate to identify the root cause and improve the implementation.

## 11. Evaluator's final decision

Candidate name: _____

Date: _____ Evaluator: _____

Total score: _____ / 100

Decision: ■ Strong hire ■ Hire with supervision ■ Further assessment ■ Do not hire

### Evidence / notes:

_____

_____

_____

### Assessment principle

A polished demo is not proof of engineering competence. The candidate should be able to explain, debug and modify their own code. This assignment evaluates the ability to build a reliable shopping agent; it does not by itself prove performance in a competitive production environment. Further work must test generalization to unseen tasks, operating cost, execution reliability and measured performance against alternatives.